

CAPSTONE PROJECT **PROPOSAL**

Title: A Decentralized Freelance Protocol with Web3 Integration

Date: 03/04/2026

April - 2026



Title: Decentralized Freelance Protocol with Web3 Integration

Date: 2026/04/03

General Information

Project Title: Decentralized Freelance Protocol with Web3 Integration

Points of Contact & Project Members

Position	Full Name	Student ID	Roles & Responsibilities (Brief)	E-mail
Supervisor	Subit Timalsina			
Team Lead	Sarun Mahrajan	0371759	Project Manager / Backend & Blockchain Developer	sarun2080-022 8@iimscollege. edu.np
Member 1	Bijee Dangol	0372053	Frontend Developer	bijee2080-212 @iimscollege.e du.np
Member 2	Pawan Poudel	0371478	Backend Database & Storage Specialist	pawan2080-02 96@iimscollege .edu.np
<i>Member 3</i>	Anushree Pradhan	0372148	Backend, API Developer, Database Assistant	anushree2080- 0344@iimscoll ege.edu.np
<i>Member 4</i>	Runa Maphu	0371373	System Workflow Engineer, Junior Backend Developer	runa2080-0189 @iimscollege.e du.np

DECLARATION

I Sarun Mahrajan hereby declare that this Final Year Project titled “A Decentralized Freelance Protocol with Web3 Integration” submitted to Taylors University, Malaysia in partial fulfillment of requirement for the Bachelors Degree in Computer Science is based on my original work. I also declare that this work has not been submitted elsewhere for any other degree or purpose.

Date: 04/04/2026

Roll no: 0371759

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to my supervisor Mr.Subit Shrestha for his full support which enabled me to focus on the research. Professor Subit and his academic guidance enriched my work greatly. Furthermore, I would also like to thank Mr. Anmol K.C. for distilling his own opinions.

To the staff, and my peers, I thank them for the encouragement and the intellectual environment crucial to complete my research.

TABLE OF CONTENT

Chapter 1 Project Introduction.....	1
1.1 Executive Summary.....	1
1.2 Project Objectives.....	1
1.3 Project Description.....	2
1.3.1 Problem Statement.....	2
1.3.2 Proposed Solution.....	2
1.3.3 Technology Stack.....	3
Table 1.1: Technology Stack Implemented.....	3
1.4 Project Scope.....	3
1.4.1 In-Scope Features.....	3
1.4.2 Out of Scope Features.....	4
1.5 Project Organization.....	4
1.5.1 Project Roles.....	4
Table 1.2: Project Roles and Responsibility Definition.....	4
1.5.2 Organizational Chart.....	5
Fig 1.1: Organizational Chart.....	6
Chapter 2 Literature Review.....	7
Fig 2.1 : Evolution of the Internet.....	7
Fig 2.2: Merkle Tree.....	8
2.1 Decentralized Finance (DeFi).....	9
2.2 Smart Contract Engineering.....	9
Fig 2.3: Smart Contracts in Blockchain.....	10
Fig 2.4: Smart Contract Engineering.....	11
2.3 Ethereum, Transactions & Essential Terms.....	12
2.4 Wallet-Based Authentication.....	13
Table 2.1: Custodial vs Non-Custodial Wallet Comparison.....	14
2.5 Challenges in Development.....	14
2.6 Research Gaps.....	14
2.7 Summary and Relevance.....	15
Fig 2.5: Implementation of System.....	15
2.8 Literature Review Table.....	16
Table 2.2: Literature Review Table.....	16
Chapter 3 System Analysis.....	19
3.1 SWOT Analysis.....	19
Fig 3.1: SWOT analysis of Solidity/ Ethereum.....	20

Fig 3.2: SWOT analysis of Rust / Solana.....	20
3.2 Requirement Specifications.....	21
Chapter 4 System Design.....	23
4.1 Sequence Diagram.....	23
4.1.1 MetaMask Authentication.....	23
Fig 4.1: MetaMask Authentication Sequence Diagram.....	24
4.1.2 Contract Creation.....	25
Fig 4.2: Contract Creation & Storage Sequence Diagram.....	25
4.1.3 Contract Execution.....	26
Fig 4.3: Contract Execution Sequence Diagram.....	26
4.1.4 Dispute Initiation.....	27
Fig 4.4: Dispute Initiation Sequence Diagram.....	27
4.1.5 Contract Dispute Resolution.....	28
Fig 4.5: Dispute Resolution Sequence Diagram.....	28
4.2 Activity Diagram.....	29
4.2.1 Metamask Authentication.....	29
Fig 4.6: MetaMask Authentication Activity Diagram.....	29
4.2.2 Contract Creation.....	30
Fig 4.7: Contact Creation Activity Diagram.....	30
4.2.3 Contract Execution.....	31
Fig 4.8: Contract Execution.....	31
4.2.4 Dispute Initiation and Resolution.....	32
Fig 4.9: Dispute Initiation and Resolution.....	32
4.3 Class Diagram.....	33
Fig 4.10: Entities for Application Class Diagram.....	33
4.4 Architecture Diagram.....	34
Fig 4.11: Architecture Diagram.....	34
Chapter 5 References.....	35
Chapter 6 Appendix.....	37

Table of Tables

Table 1.1: Technology Stack Implemented	3
Table 1.2: Project Roles and Responsibility Definition	4
Table 2.1: Custodial vs Non-Custodial Wallet Comparison	14
Table 2.2: Literature Review Table	16

Table of Figures

Fig 1.1: Organizational Chart.....	6
Fig 2.1 : Evolution of the Internet.....	7
Fig 2.2: Merkle Tree.....	8
Fig 2.3: Smart Contracts in Blockchain.....	10
Fig 2.4: Smart Contract Engineering.....	11
Fig 2.5: Implementation of System.....	15
Fig 3.1: SWOT analysis of Solidity/ Ethereum.....	20
Fig 3.2: SWOT analysis of Rust / Solana.....	20
Fig 4.1: MetaMask Authentication Sequence Diagram.....	24
Fig 4.2: Contract Creation & Storage Sequence Diagram.....	25
Fig 4.3: Contract Execution Sequence Diagram.....	26
Fig 4.4: Dispute Initiation Sequence Diagram.....	27
Fig 4.5: Dispute Resolution Sequence Diagram.....	28
Fig 4.6: MetaMask Authentication Activity Diagram.....	29
Fig 4.7: Contact Creation Activity Diagram.....	30
Fig 4.8: Contract Execution.....	31
Fig 4.9: Dispute Initiation and Resolution.....	32
Fig 4.10: Entities for Application Class Diagram.....	33
Fig 4.11: Architecture Diagram.....	34

Chapter 1 Project Introduction

1.1 Executive Summary

The project involves implementation of a decentralized storage application with blockchain technology to create a freelancing web application which offers secure, transparent and a trustless system for the clients and freelancers involved. The platform implements digital wallets where user identity is pseudonymously secure and stored only on sessions to ensure best privacy for individuals while maintaining blockchain transparency for all transactions. Furthermore, contract terms and deliverables are stored in an IPFS to ensure data redundancy and safety. The system eliminates traditional middlemen/ centralized authorities who take a portion of all transactions as customary fee, replacing them with securely held and automatically released smart contracts for all ether managed in the transaction. The transactions and executable logic along with escrow is handled by the Ethereum Blockchain.

The project aims to implement a decentralized freelancing platform where users authenticate via MetaMask, create and deploy their smart contracts on the blockchain, submit and store deliverables in IPFS, and receive payments according to contracts and the created logic whilst removing a centralized organization.

1.2 Project Objectives

Primary Objectives for the freelancing application are:

- Implement Decentralized Marketplace: The use of Ethereum Blockchain with Decentralized File Storage to enable direct client-freelancer transactions.
- Implement Smart Contracts: The use of Solidity to create executable smart contracts which are stored in GoEth; handling payment and deliverables.
- MetaMask Authentication: Use wallet-oriented system for identity checking as well as digital signature for transactions.

- Develop Web3 accurate applications: Develop off-chain and on-chain hybrid system to remove central authorities and middlemen.
- Develop User Friendly Interface: Build simple, intuitive UI using React Framework whilst integrating mentioned IPFS, MetaMask, Blockchain and Smart Contract generation.

1.3 Project Description

1.3.1 Problem Statement

Currently, the most popular freelancing platforms such as Upwork, Fiverr, Freelancer.com, etc, are all centralized applications and charge substantial fees up to 20% of the project and its deliverables. Considerable pain points for these platforms include Payment Delays, Lack of Transparency, Lack of Dispute Resolution and users sensitive data privacy concerns.

1.3.2 Proposed Solution

Accordingly, the decentralized freelancing application addresses the challenges through:

- Smart Contract Execution: Funds are stored in Ethereum smart contracts and are released based upon previously defined milestones or approved by clients.
- Transparent Design: Transactions, payments, contract state changes are all viewable on the blockchain which also provides immutable audit trails.
- Privacy First: Users' wallet data is never stored in databases, they are only identified through pseudonymous ID (eg: usr_abc123).
- Dual Storage: Smart contracts terms and deliverables are stored on IPFS to ensure redundancy and immutability, whilst smart contracts are stored and executed on the blockchain.
- Reduced Fees: Platform operates without a centralized authority so transaction charges only depend upon Gas Fees.

1.3.3 Technology Stack

The various tools, frameworks, languages, environments, databases required for the application include:

Table 1.1: Technology Stack Implemented

Layer	Technology	Version
Frontend Framework	React	18.2.x
Frontend Language	JavaScript	6.x
Backend Framework	FastAPI (Python)	0.100+
Database	PostgreSQL	15.x
Session Store	Redis	7.x
Smart Contracts	Solidity	0.8.x
Contract Testing Environment	HardHat	2.x
IPFS Implementation	Kubo	0.28.x
Wallet	MetaMask	BrowserExtention

1.4 Project Scope

1.4.1 In-Scope Features

Features which will be included in the final version of the decentralized application include:

- MetaMask Authentication: EIP-712 signed messages for verification and secure login without traditional passwords.
- Contract Creation: Clients and Freelancers define their own contracts depending upon their project. Payments, deliverables, terms are defined by them.

- **Contract Execution:** Track project deliverables, milestones in the contract and execute logic depending upon them.
- **IPFS Deliverable Storage:** Project deliverables stored in the decentralized file storage for future audit.
- **Dispute Resolution:** Mechanisms for clients or freelancers to raise disputes and handle said disputes.

1.4.2 Out of Scope Features

Features which will not be included in the final version of the project include:

- **Cross System Operability:** Web based application not suitable for mobile, tablets, etc.
- **Fiat Currency Integration:** Ether or ETH tokens only.
- **Multi-Chain Support:** Ethereum mainnet/ testnet only.
- **Post Deployment Smart Contract upgrades.**
- **Traditional Payment Gateway Implementation.**
- **Real Time VoIP.**

1.5 Project Organization

The project implements Agile methodology with a biweekly sprint planning to effectively communicate, review and develop the application.

1.5.1 Project Roles

Table 1.2: Project Roles and Responsibility Definition

Name	Role	Responsibility
Sarun Maharjan	Project Manager / Contract And Blockchain Developer	- Coordinate team activity - Design Smart Contract - Implement MetaMask Authentication

		- Implement Ethereum Blockchain
Anushree Pradhan	Backend & API Developer	- Implement FastAPI to develop RESTful API - Handle Server side Logic - Integrate Backend services with blockchain - Integrate PostgreSQL
Bijee Dangol	Frontend Developer	- Develop web interface - Integrate Blockchain Interaction with visuals - Manage User Experience
Pawan Poudel	Database & IPFS Storage	- Assist in PostgreSQL indexing - Implement IPFS storage
Runa Mapu	System Workflow Engineer & Junior Backend Developer	- Design and develop role specific system workflow - Assist in API integration

1.5.2 Organizational Chart

The project is role-based with overlapping duties, hence, particular individuals have leadership in different areas.

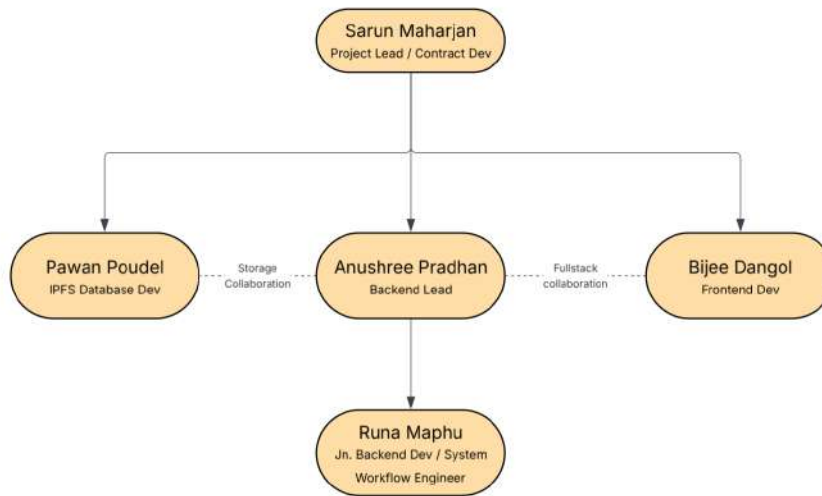


Fig 1.1: Organizational Chart

Chapter 2 Literature Review

Ever since the invention of the World Wide Web, the characteristics of the data transfer available, the medium through which data has been transferred, and the amount of user generated content has indicated the generation of the web currently ongoing. Currently, the web has been divided into three generations:

- Web 1.0 (1989 - 2001): “Read-only” era of web with limited information exchange and static pages.
- Web 2.0 (2004 - Present): “Read-Write” era of web with dynamic pages, user-generated content (UGC), and social exchange.
- Web 3.0 (2006 - Present): Enhanced data resource access through interlinking between websites. (Lai et al., 2023)

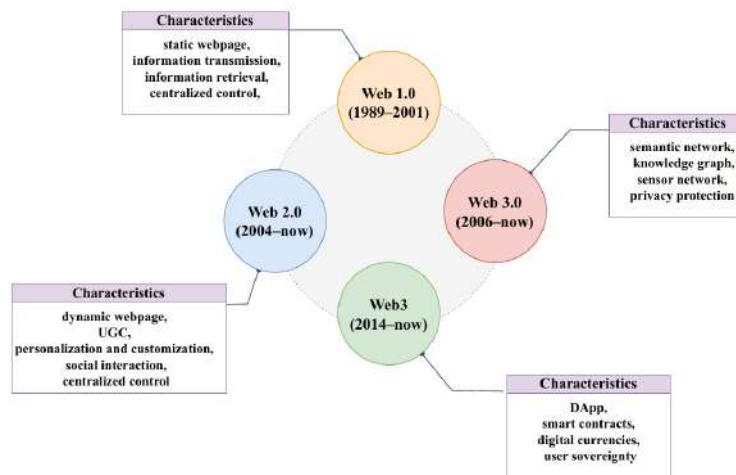


Fig 2.1 : Evolution of the Internet

- Web3 (2014 - Present):

“Read-Write-Own” era of web proposed by Gavin Wood in 2014, utilizing blockchains and cryptocurrency to establish a decentralized network with decentralized storage application, Smart Contracts and Decentralized Applications (DApps). Some features which distinguish Web3 from other eras of Web are Immutability, Trustless feature (no reliance on third party), user anonymity, and data ownability. (Huang et al., 2024 & Lai et al., 2023)

The main technologies associated with Web3 include components such as:

1. Blockchain:

Blockchains play a crucial role as it possesses decentralized characteristics and the ability to restore data ownership to users. Validated transactions are aggregated into blocks and added to the growing blockchain through consensus mechanisms. (Lai et al., 2023)

2. Digital Signature:

Digital Signatures are essential for identity verification and authentication, data protection as well as tamper resistance. Initially, upon generating the digital signature, the sender uses a hash operation to convert data to hash code and encrypt it via their private key then sending both ciphertext and hash value to the recipient. Upon receiving the ciphertext, the recipient uses the sender's public key to decode the ciphertext and compare the decoded hash with received hash. If the hash remains consistent then we can infer that the sender's data remains intact and their identity is reliable. (Lai et al., 2023)

3. Merkle Trees

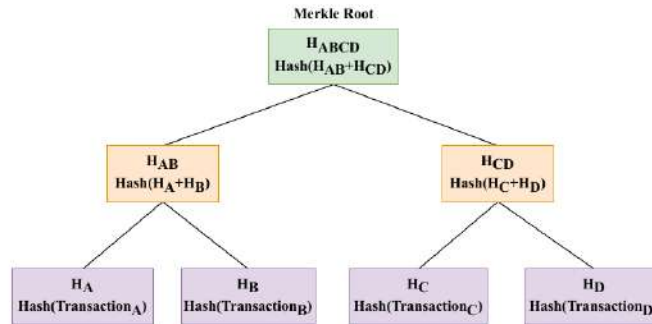


Fig 2.2: Merkle Tree

A merkle tree is a binary tree which stores the hash value in its nodes, with each leaf node representing a hash value/ an individual transaction. Merkle trees are created to ensure that any minor alteration in the hash value for any transaction will trigger substantial modifications in the entire tree. This also allows any node downloading block information to verify all transactions prior, ensuring data integrity. (Lai et al., 2023)

These trees are created through consistently hashing the hash values of each pair of nodes together to create a new hash value. The new hash values are stored in a new node and repeated until there is only a single hash value (Merkle Root) which represents all the transactions within this Tree. (Lai et al., 2023)

4. Smart Contracts

Smart contracts are modular, reusable, and automatically executed scripts which enable users to establish agreements and obligations. The contracts themselves are deployed into the blockchain as code, utilizing commands to invoke the smart contract. (Lai et al., 2023)

2.1 Decentralized Finance (DeFi)

DeFi refers to financial systems which run on top of the blockchain. It is designed to simplify token-swapping through standard sets of APIs or SDKs. It can further be subdivided into smaller categories:

- Trading/ Exchange: Token (ether) swapping without a central governing agency.
- Investment: Decentralized investment smart contract in which users can deposit funds and create smart contracts which are executed after a set of predefined rules are in effect.
- Wallets: Digital tools which enable users to manage their assets and interact with Web3 applications. (Huang et al., 2024)

2.2 Smart Contract Engineering

Smart contracts; proposed by Nick Szabo in 1994 are algorithmic specifications for specific scenarios where Szabo himself used a car loan scenario where: if the lender fails to pay their loan, the smart contract would automatically withdraw their digital car keys. (Hu et al., 2020)

Furthermore, with the rise of blockchain, its integration with smart contracts gives flexibility to design and develop and implement real world problems. These smart contracts remain as scripts on the blockchain with an ability to be executed upon a trigger mechanism. (Mohanta et al., 2018)

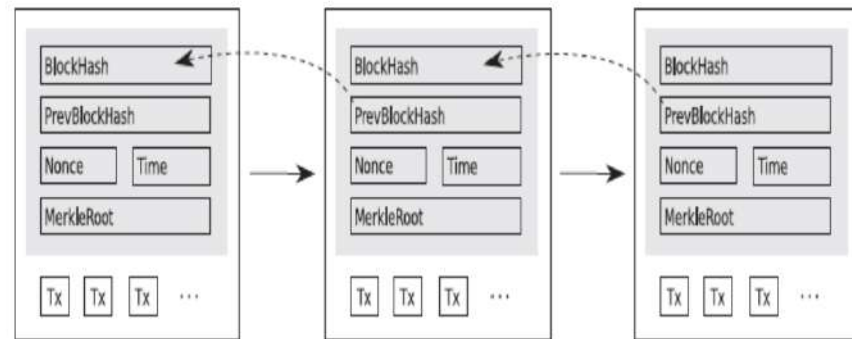


Fig 2.3: Smart Contracts in Blockchain

“Smart Contract Engineering is a systemized, modularized, and judgemental process for the smart contract that is based on development, maintenance, and execution, and that integrates with software engineering, intelligent methods as well as legal code technology.” - (Hu et al., 2018)

The core parts of contract engineering are:

1. Informal contract specifications are mapped with accordance to both parties.
2. Formal Description of the contract is created without ambiguity.
3. Contract law regulation will ensure consistency of the contract with existing laws so records on the blockchain can be used as evidence in judicial proceedings.
4. Smart contracts are logically verified using contract verification techniques.
5. Contracts are modelled mathematically for contract code generation.
6. Conformance testing determines whether code implemented meets all specified formal requirements.

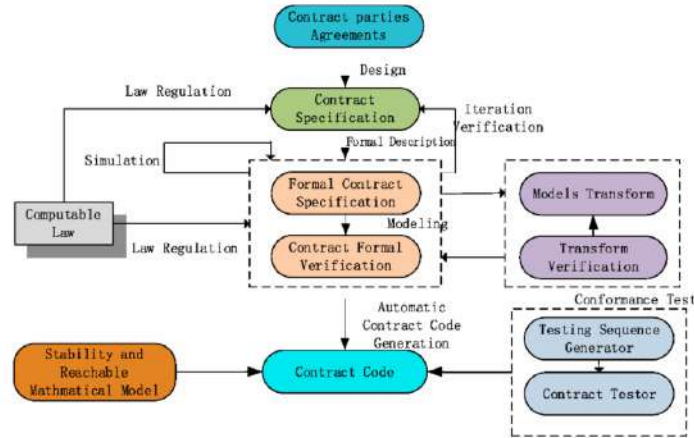


Fig 2.4: Smart Contract Engineering

The underlying architecture of its assimilation with blockchain comprises two main layers i.e. 1) The consensus Layer and 2) The contract Layer.

The consensus layer lies on top of the data and network services of the underlying architecture. It sets rules and regulations for any contract before its participation with the underlying platform. It includes a protocol for sharing the transactions across the network and grouping the transactions into blocks. Furthermore, the shared data structure (mempool) is ordered into nodes, the various nodes are then ordered according to a set of protocols which chooses the next block in the sequence known as blockchain. The blockchain itself is also seen as a global state machine and is replicated at all consensus nodes. Finally, the consensus layer is responsible for the safety, liveness, and finality of the blockchain. (Bartoletti et al., 2025)

The contract layer sits on top of the consensus layer and hosts the execution environment for each smart contract. As the consensus layer is responsible for the addition of new blocks, the execution of each transaction is observed in the contract layer and observes its interactions with the environment and other nodes already present within the blockchain (Bartoletti et al., 2025). It would handle pseudocode such as:

```

contract Escrow {
    enum State { Open, Funded, Resolved, Disputed }
    address public client = msg.sender;
    address public freelancer;
    State public state;

    event Update(State nState, uint val);

    constructor(address _f) { freelancer = _f; }

    function fund() external payable {
        require(msg.sender == client && state == State.Open && msg.value > 0);
        state = State.Funded;
        emit Update(state, msg.value);
    }

    function release() external {
        require(msg.sender == client && state == State.Funded);
        state = State.Resolved;
        payable(freelancer).transfer(address(this).balance);
        emit Update(state, 0);
    }

    function dispute() external {
        require((msg.sender == client || msg.sender == freelancer) && state ==
State.Funded);
        state = State.Disputed;
        emit Update(state, 0);
    }

    function refund() external {
        require(msg.sender == client && state == State.Disputed);
        state = State.Resolved;
        payable(client).transfer(address(this).balance);
        emit Update(state, 0);
    }
}

```

2.3 Ethereum, Transactions & Essential Terms

Ethereum is a decentralized, open-source blockchain platform with an intent to improve scripting, on-chain protocols and to assist developers in creating applications involving Web3 development. It provides a built-in programming language, which allows for any individual to create smart contracts as the language itself is also a high-level language with complex rules for ownership, transactions and state functions. Here,

‘ether’ is used as the main crypto-currency for all transactions taking place. (Buterin, 2013)

- Transactions

Transactions refer to signed data packages which store messages to be sent to another account. Furthermore, it also consists of the sender's digital signature, the amount of ether used for the sending of the message and two values STARTGAS and GASPRICE.

Here, the STARTGAS refers to the number of lines of code which can be executed in any transaction. This is to ensure there is no infinite looping. Similarly, GASPRICE refers to the fee which needs to be paid for the use of the application for transactions.

- Fees

As transactions require to be published into the blockchain, fees are established to prevent abuse through purely voluntary fees. Here, although transactions may appear to be born from a single account, as every transaction needs to be processed by every node in the consensus layer, the vast majority of the cost is borne from third parties.

2.4 Wallet-Based Authentication

Digital tools which allows users to manage their cryptocurrencies and interact with Web3 and Web3 applications; wallets can be divided into Custodial and Non-Custodial Wallets both with varying features and uses.

Non-Custodial wallets in particular have a niche feature which allows for authentications through the use of holders' private keys (Seymour & Goodell, 2024).Some key differences between wallets to take into account are:

Table 2.1: Custodial vs Non-Custodial Wallet Comparison

Criterion	Custodial	Non-Custodial
Key Control	Service provider	User exclusively
Recovery	Password reset, support	Seed phrase backup
Security Model	Trust in provider	User responsibility
Platform Dependency	High	Low
User Experience	Traditional feel	Requires crypto knowledge
Counterparty Risk	Provider failure/lockup	None

MetaMask; another non-Custodial wallet has gained popularity and is the dominant Ethereum Wallet for Web3 applications. It is available as an extension, application and website to serve as the primary bridge between users and ethereum blockchain.

2.5 Challenges in Development

As the infrastructure is an emerging field, problems in the blockchain could severely limit the development. Some aspects which impact the Web3 ecosystem and its development include (Huang et al., 2024):

- **Interoperability:** Communication and data transfer between various blockchains is not available, restricting the flow of ether between various Dapps.
- **Code Security:** Web3 applications are open source meaning hackers can easily point exploits and launch attacks.
- **Privacy:** Data stored on blockchain are publicly visible, making users more vulnerable, especially if wallet and physical identity is revealed.

2.6 Research Gaps

Throughout the literature review, several gaps have been identified such as:

- Limited research on implementation of **wallet based authentication**
- Insufficient implementation of blockchain with decentralized storage

- Overemphasis on either fully centralized or decentralized systems
- Insufficient guidelines on blockchain abstractions behind interfaces

2.7 Summary and Relevance

The reviewed papers suggest that although theoretical knowledge and advances in the field have been progressing at a rapid rate, adequate implementation of such theoretical resources has not yet been processed.

The main architecture of the system can be depicted through 1) Signature Verification within client interface through digital wallets, 2) Initiate transactions/ smart contract request, and 3) Transmit transactions into web server and block chain.

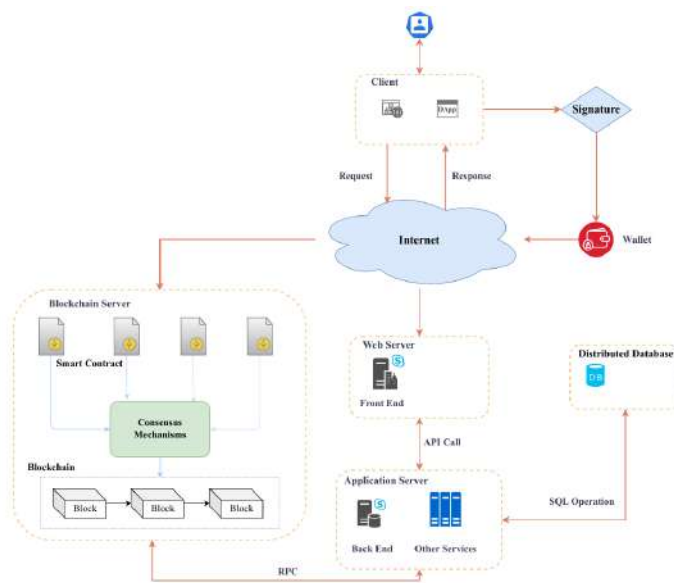


Fig 2.5: Implementation of System

Hence, insights from existing research papers directly address the design of the decentralized gig system.

2.8 Literature Review Table

Table 2.2: Literature Review Table

Author(s) & Year	Source / Context	Methodology	Key Findings	Relevance to Decentralized Gig System
Lai et al. (2023)	Web3: Exploring Decentralized Tech & Applications	Thorough exploration / Review	Web3 reshapes internet infrastructure to achieve autonomous digital experiences and ownership.	Foundational theory for the "Introduction to Web3" section.
Huang et al. (2024)	Overview of Web3: Infrastructure & Applications	Empirical study (200 projects + GitHub data)	Categorizes Web3 apps and identifies core challenges like interoperability and privacy.	Directly addresses "Challenges in Development" and DeFi applications.
Hu et al. (2020)	Smart Contract Engineering	Systematic Process Mapping	Proposes Smart Contract Engineering (SCE) to unify software engineering with legal code.	Defines the structured development process for your system's "Contract Layer".

Mohanta et al. (2018)	Overview of Smart Contract Use Cases	Literature Review & Component Analysis	Smart contracts are tamper-resistant, self-executing, and offer low-cost real-time tasks.	Supports the integration of smart contracts for gig fulfillment and payment.
Bartoletti et al. (2025)	Smart contract languages: Comparative analysis	Comprehensive Language Assessment	Analyzes strengths and weaknesses of languages (like Solidity) relative to their platforms.	Informs the technical choice of programming languages for the system's core logic.
Buterin (2013)	Ethereum White Paper	Technical Proposal	Introduced a Turing-complete blockchain allowing for any decentralized application.	Provides the fundamental architectural basis for the entire gig platform.
Seymour & Goodell (2024)	Custodial vs. Non-Custodial Wallets	Conceptual Comparative Analysis	Non-custodial wallets offer full user control and privacy but require high user responsibility.	Supports the implementation of MetaMask as the primary user authentication bridge.

McCabe et al. (2024)	Blockchain-Based Authentication Mechanism	Implementation & Security Testing	Blockchain-based 2FA methods provide superior security over conventional password systems.	Directly supports the "Wallet-Based Authentication" security model.
He et al. (2020)	Smart Contract Vulnerability & Security Audit	Survey of vulnerabilities & audit tools	Compares mainstream audit tools and identifies common Ethereum-specific security flaws.	Essential for addressing "Code Security" and auditing the Escrow contract.
Tantikul & Ngamsuriyaroja (2019)	Exploring Vulnerabilities in Solidity	Comparative Tool Analysis	Identifies flaws in decentralized programs and analyzes detection tool effectiveness.	Provides practical insights for securing Solidity code against financial exploits.
Tyurin et al. (2019)	Languages for Safe Smart Contract Programming	Comparative Analysis for Safety	Discusses language features that minimize vulnerabilities and enhance programming safety.	Guides the selection of safe programming practices for the Escrow smart contract.

Chapter 3 System Analysis

The current system proposes the use of the following tool stack for Web3 and Blockchain implementation:

- Geth (Go Ethereum): As mentioned in section 2.4, Ethereum is an open source blockchain which provides tools for easy development of Dapps. Go Ethereum is the command line interface tool for ethereum blockchain allowing for computers to create and manage their own blockchains.
- Solidity: High level - object oriented programming language which allows users to create smart contracts according to the users requirements. With notable features such as self-executing code upon certain trigger activation.
- MetaMask: As mentioned in section 2.5, MetaMask is a Non-Custodian wallet meaning it has the ability to provide authentication without requiring traditional user inputs. Furthermore, it allows users to securely manage their transactions, digital signatures, and interact with Dapps .
- Ethers.js: A javascript library essential in communication between the blockchain and the application. It is responsible for sending and receiving data and transactions, as well as interacting with deployed smart contracts.

3.1 SWOT Analysis

To understand the strengths of the chosen tools, it is important to note that Solidity is the primary language for the Ethereum blockchain whilst Rust is the primary language for Solana. Although both Ethereum and Solana can support other languages such as Vyper, Flint, Bamboo, etc; Solidity and Rust are the most widely used for the already present support and development tools.

1. Solidity / Ethereum



Fig 3.1: SWOT analysis of Solidity/ Ethereum

2. Rust / Solana



Fig 3.2: SWOT analysis of Rust / Solana

Hence, Solidity is deemed the better blockchain technology with solidity being the adequate programming language.

3.2 Requirement Specifications

The functional and non-functional requirements for the system are:

1. Functional Requirements:

To meet the functional requirements according to the role of Blockchain, Smart Contract and MetaMask application developer; the requirements are:

a. Smart Contract Development and Deployment

- Users should be able to develop and deploy smart contracts to the Ethereum Blockchain.
- Smart Contracts must be monitored with their states i.e. In-Progress, Released, Dispute, or Refund.

b. Wallet Integration

- MetaMask should be a viable tool to authenticate into the system.
- Smart Contracts can be signed via digital signatures using MetaMask.

c. Role Based Access

- Individuals or accounts with whom the smart contract is tied to can only view the smart contract.

d. Transactional Functionalities

- Deployed Smart contracts must be executable according to pre-requisites conditions.
- Transactions such as success, deadline met, or decline procedures (from clients) must be handled.

e. Dispute Initiation and Resolution

- Client and Freelancer both should be able to initiate a dispute.
- Admin must be able to initiate resolution with either refunding client or proceeding to pay freelancer the remaining frozen ether in smart contract.

2. Non-Functional Requirements:

Concurrently, the non-functional requirements for the application according to the defined functional requirements are:

- a. Performance
 - Wallet authentication requests must be handled under 5 seconds.
 - Data Transfer from frontend to blockchain must be handled in under 3 seconds.
- b. Compatibility
 - MetaMask extension and browser compatibility.
- c. Blockchain Security
 - Ensure fault tolerance for blockchain transaction failures.
- d. Transparency
 - Contracts must be viewable to those with contract signatures.
- e. Privacy
 - User's wallet data should not be stored in off chain databases.

Chapter 4 System Design

Designing the various aspects of the working mechanisms of the system, simple yet structured visualizations of the decentralized application are created below with the assistance of plantUML and graphviz.

4.1 Sequence Diagram

Multiple sequence diagrams for the various functionalities of the created dapp include;

4.1.1 MetaMask Authentication

The authentication sequence diagram demonstrates how the system handles login via Ethereum Wallet (MetaMask). The system initially attempts to inspect whether the individual has a MetaMask account, only approving log-in after the digital signature verification.

Upon signature verification, the system assigns the user a "session" of 1 hour. The session consists of the user's wallet, role and pseudonym which can be used for further transactions. Finally, either upon log out or an hour passes, the session is deleted.

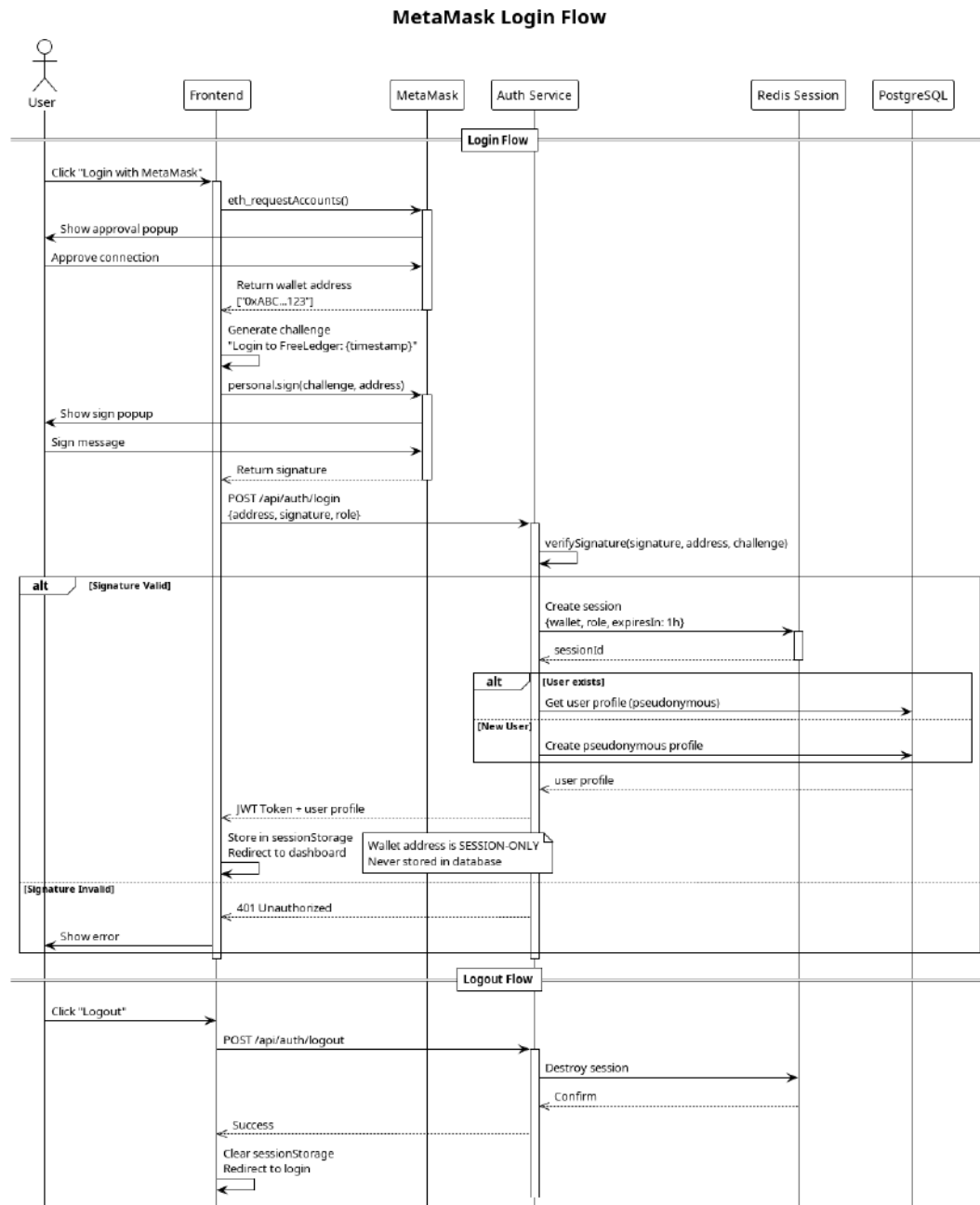


Fig 4.1: MetaMask Authentication Sequence Diagram

4.1.2 Contract Creation

Contract creation or job creation requires the client to create the smart contract filling in details such as payment, required deliverables, deadlines, additional remarks, etc. The client then signs the created contract prior to storing the contract in IPFS. Upon uploading to Kubo, it generates the uploaded contents CID, the CID along with contracts metadata is stored in PostgreSQL. Upon uploading to Kubo, it generates the uploaded contents CID, the CID along with contracts metadata is stored in PostgreSQL.

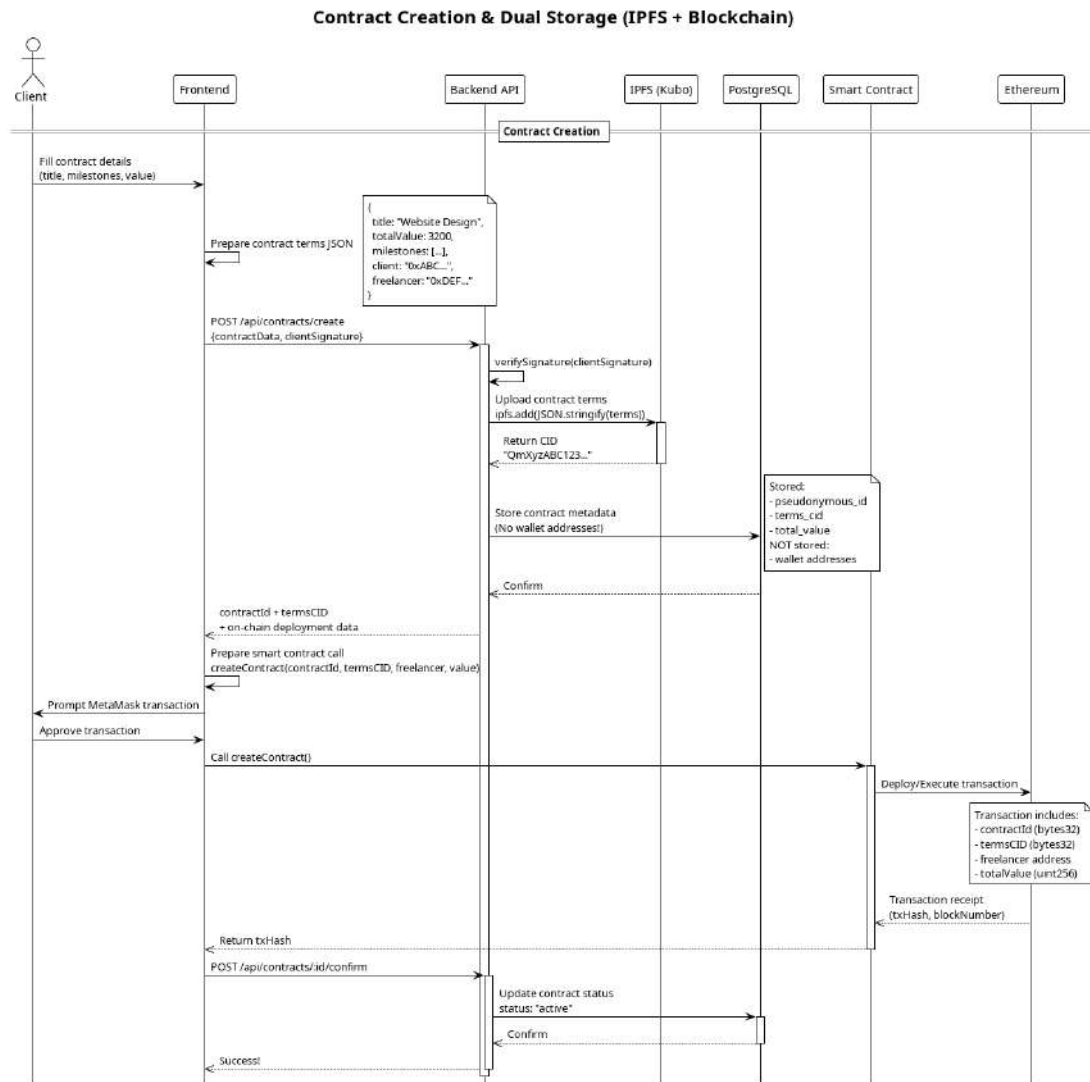


Fig 4.2: Contract Creation & Storage Sequence Diagram

4.1.3 Contract Execution

Freelancers Upload deliverables to IPFS, once the deliverables have been uploaded, its CID is stored in postgresSQL and upon all deliverables being reached clients then gain the ability to approve the deliverables and approve whether for the milestone the freelancer is paid.

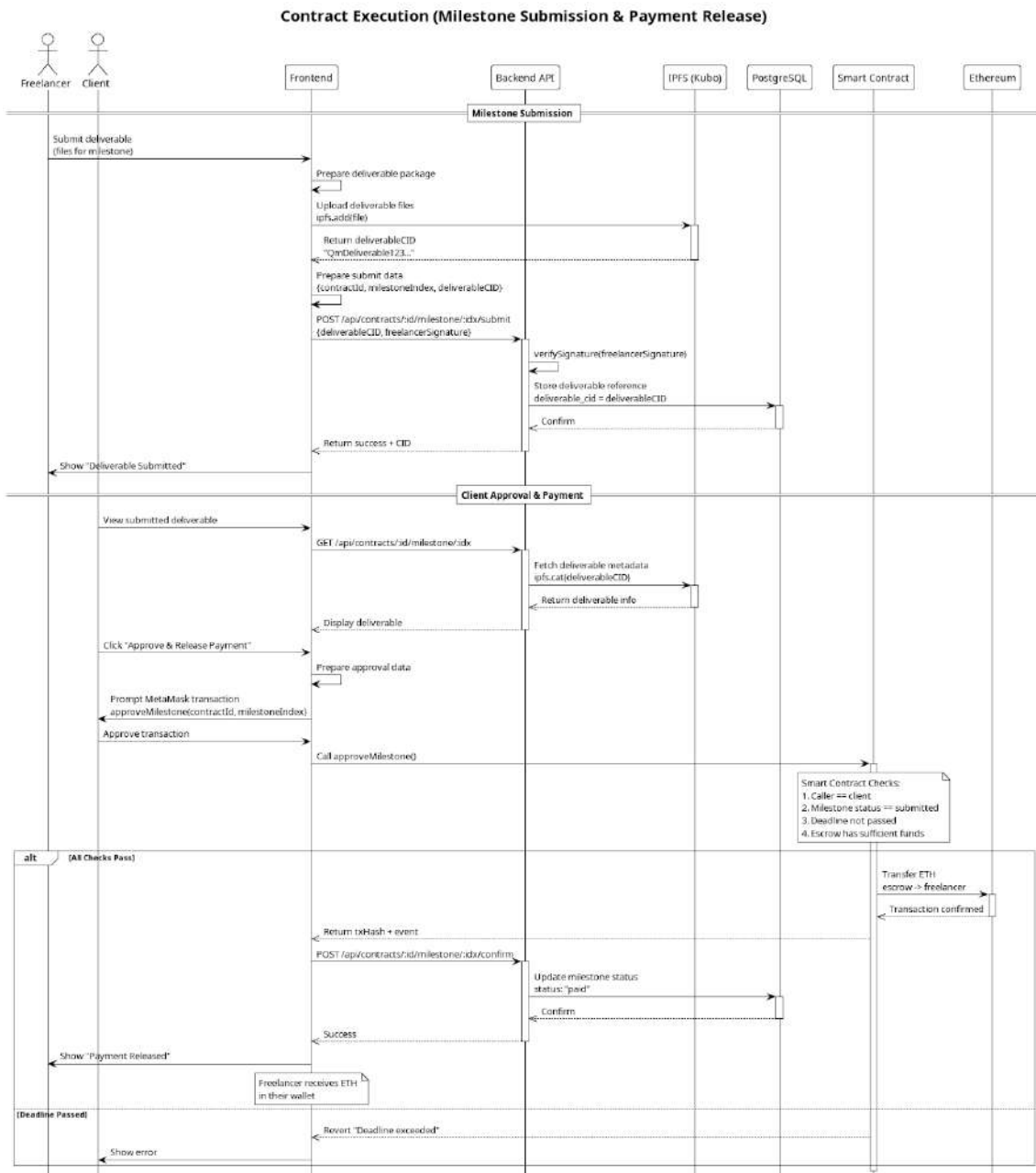


Fig 4.3: Contract Execution Sequence Diagram

4.1.4 Dispute Initiation

Either Client or Freelancer can raise a dispute regarding the project. Here, an admin is required to initiate the review process, where they fetch dispute, metadata of project, smart contract from Kubo using CID obtained from postgresQL, along with deliverables to make a decision.

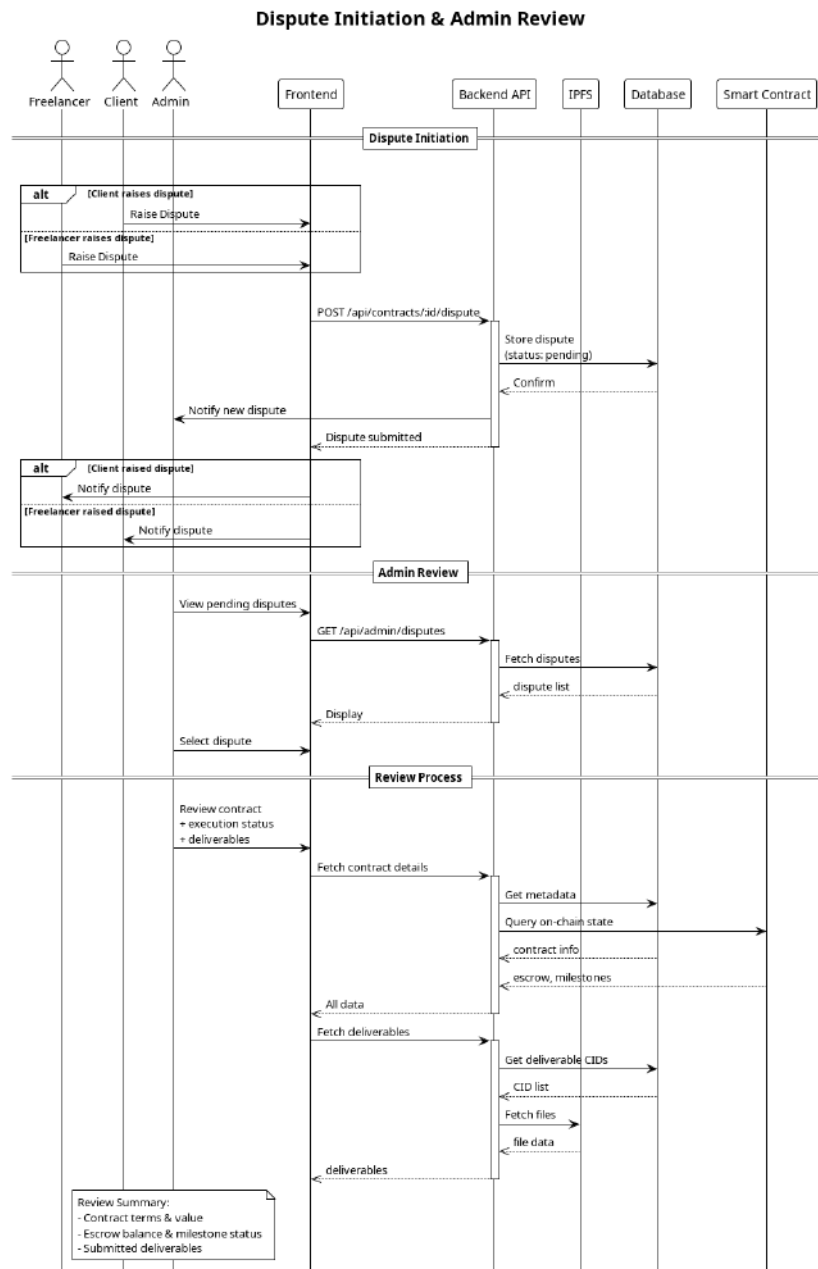


Fig 4.4: Dispute Initiation Sequence Diagram

4.1.5 Contract Dispute Resolution

To resolve the dispute, admin monitors all the deliverables, reads the contract, observes the metadata, and decides whether to transfer frozen ether to freelancer or to client. The process for ether transfer depends upon the inbuilt smart contract function `resolveDispute(contractID,X)`. Upon transferring the ether, projects dispute status is updated and maintained.

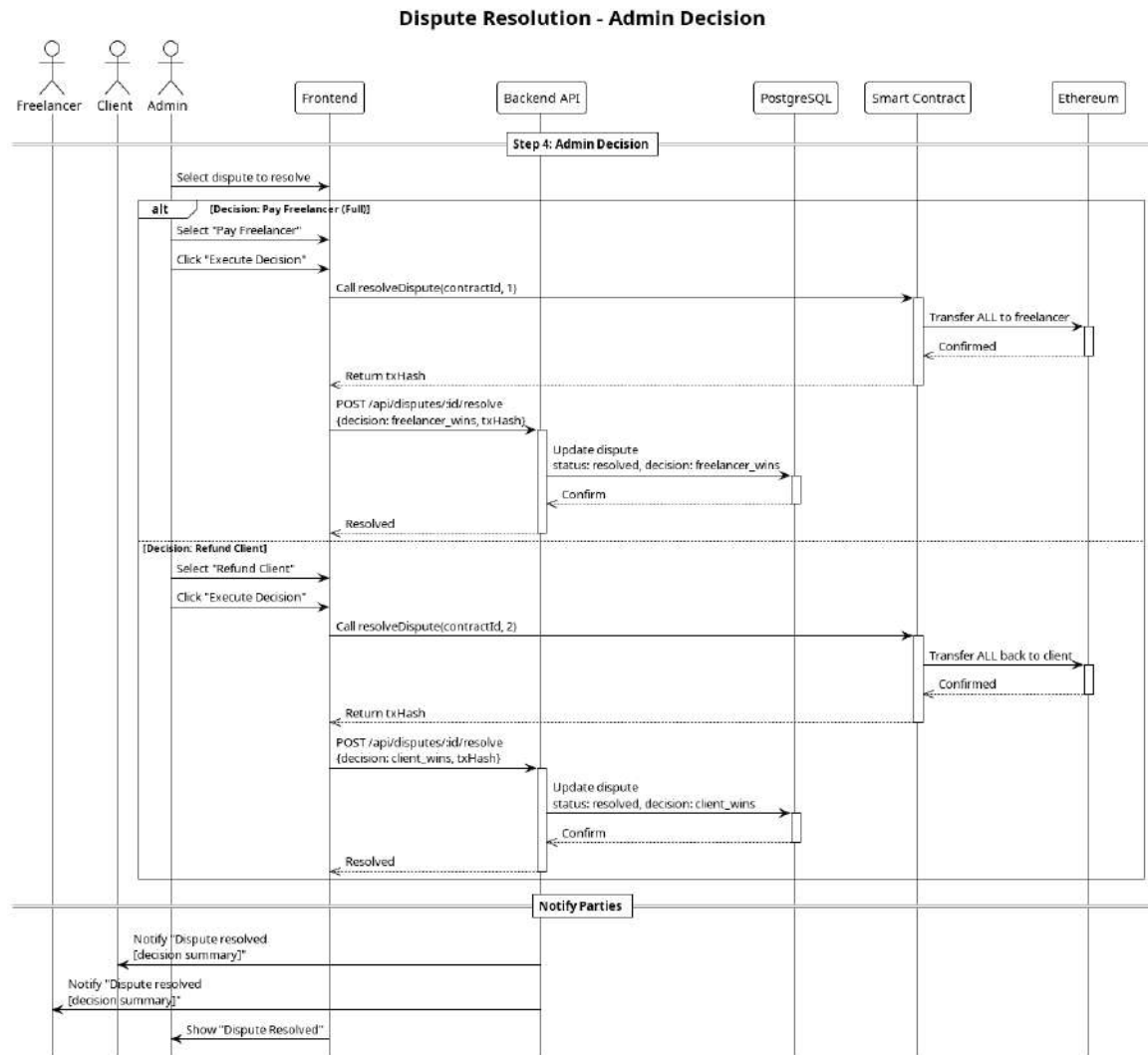


Fig 4.5: Dispute Resolution Sequence Diagram

4.2 Activity Diagram

4.2.1 Metamask Authentication

Authentication begins with connecting to metamask where metamask generates the user's digital signature as a sign message. The time for login is stored for JWT sessions so each session can last for an hour. If the received signature is deemed valid, redis creates the session for an hour and the user is sent to their appropriate dashboard.

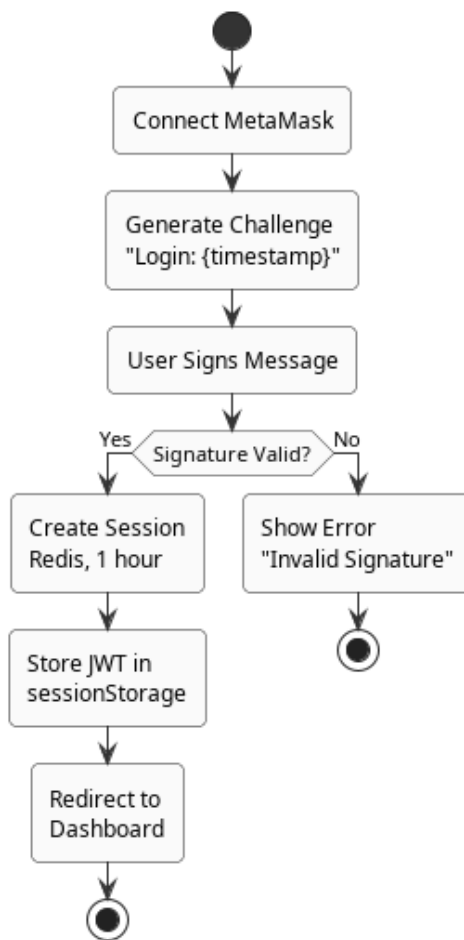


Fig 4.6: MetaMask Authentication Activity Diagram

4.2.2 Contract Creation

Contract creation begins with clients filling up the details regarding particular gigs, after they are satisfied, solidity generates the contract depending upon their requirements. The created contract is then uploaded to Kubo whilst data for postgresSQL is also prepared (eg: wallet, pseudonymous user ID, etc.). The CID is then coupled with postgresSQL and stored concurrently. The smart contract status is deemed pending until freelancer and client signs through MetaMask. Upon signature verification, the Contract is stored as well as the ether.

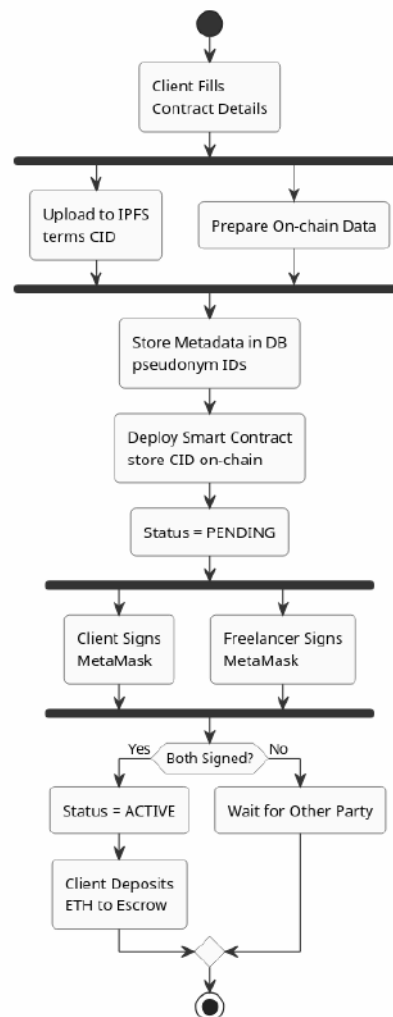


Fig 4.7: Contact Creation Activity Diagram

4.2.3 Contract Execution

Freelancer delivers the required files in IPFS, the files CID is returned and stored in the central postgresQL database and the client is informed. The client can now download the deliverables and approve of the deliverables or request for changes. Upon accepting the deliverables, ether is transferred to the client.

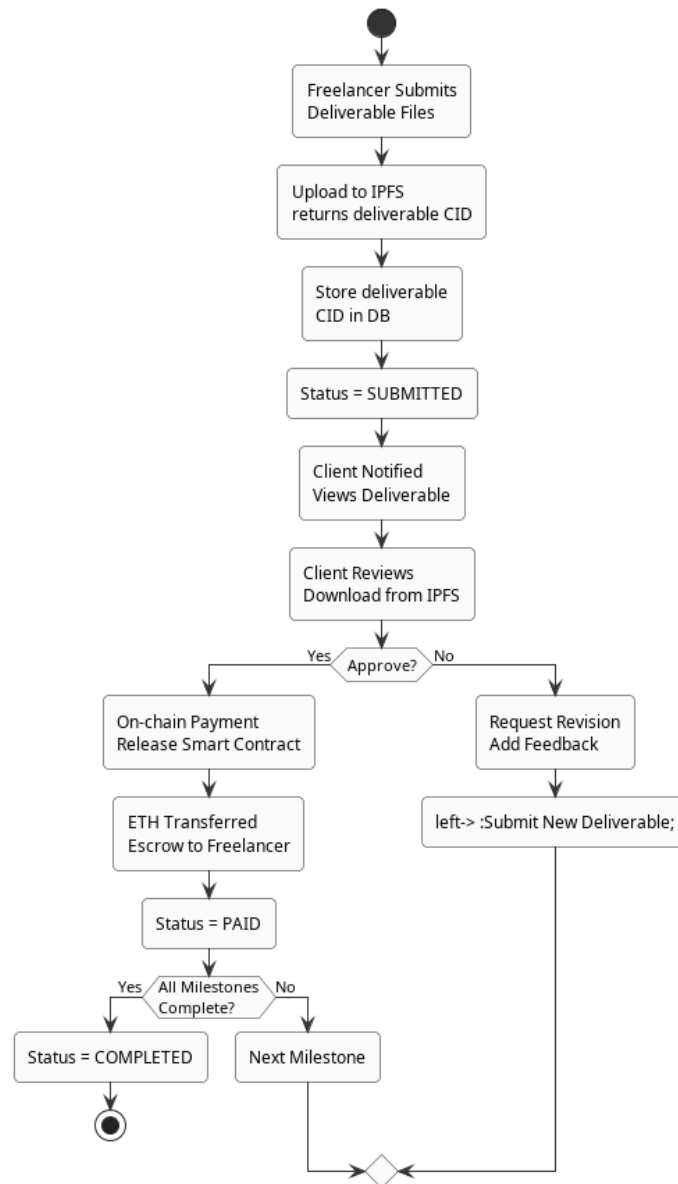


Fig 4.8: Contract Execution

4.2.4 Dispute Initiation and Resolution

Either freelancer or client can raise a dispute which is stored in the off-chain database. The corresponding client or freelancer whose signature is shared in the smart contract is notified as well as the dispute dashboard updated.

Dispute Initiation & Resolution Activity Diagram

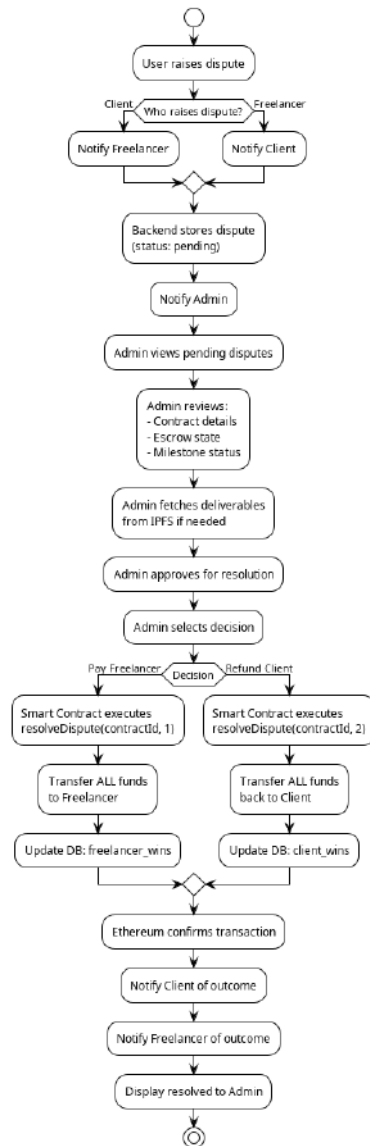


Fig 4.9: Dispute Initiation and Resolution

Admins can now address the dispute by reviewing the smart contracts, the deliverables and milestones. Finally the admin decides whether the frozen smart contract and the ether

let is provided to the freelancer or refunded to the client. Finally, both the parties are notified about the decision taken.

4.3 Class Diagram

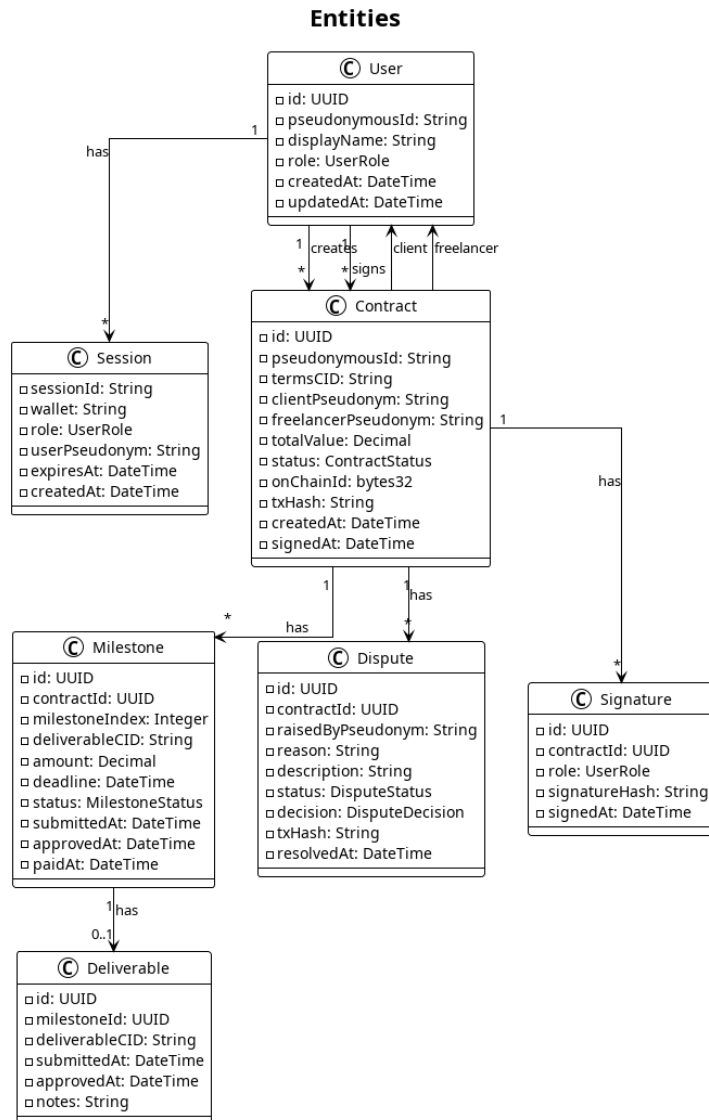


Fig 4.10: Entities for Application Class Diagram

The class diagram consists of the following primary entities; Users, Contract, Session and Deliverable. The User can be of different roles such as client, freelancer or admin whilst the contract is created by the client with varying degree of terms. The contract

must include milestone definitions as well prior to its signing, the mechanism for signing or the digital signature itself is within the session. However, for each contract the signature is unique as well to further ensure security.

Upon Contract creation with the various roles of the Users, along with their digital signatures in the session. The deliverables are also mapped to the contract and the milestones within the contract. Finally, in case of disputes the dispute class is called as a storage mechanism.

4.4 Architecture Diagram

The architecture diagram revolves around smart contract creation, with ether.js being the medium through solidity can coordinate with the frontend, the system itself first requires creation of a user session through redis, upon which they can now create the contract - sign the contract- then store the contract and its metadata in off-chain and on-chain databases respectively.

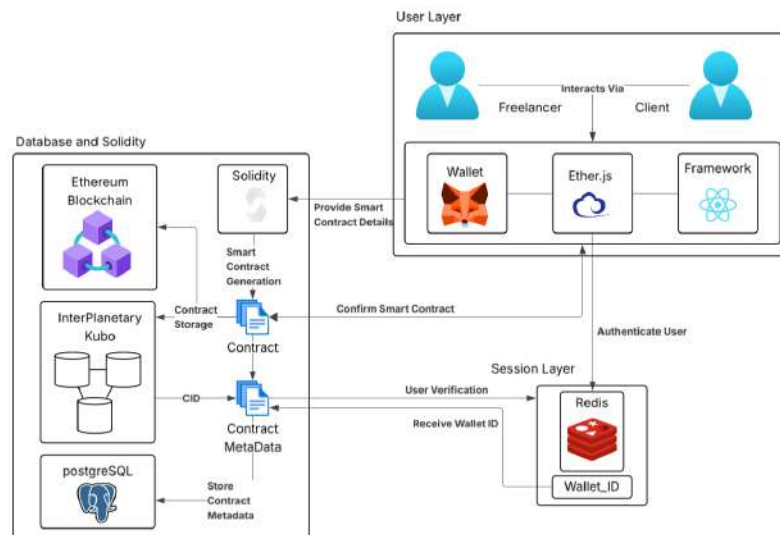


Fig 4.11: Architecture Diagram

Chapter 5 References

- Bartoletti, M., Benetollo, L., Bugliesi, M., Crafa, S., Dal Sasso, G., Pettinau, R., Pinna, A., Piras, M., Rossi, S., Salis, S., Spanò, A., Tkachenko, V., Tonelli, R., & Zunino, R. (2025). Smart contract languages: A comparative analysis. *Future Generation Computer Systems*, 164, 107563. <https://doi.org/10.1016/j.future.2024.107563>
- Fiorentino, S., & Bartolucci, S. (2021). Blockchain-based smart contracts as new governance tools for the sharing economy. *Cities*, 117, 103325. <https://doi.org/10.1016/j.cities.2021.103325>
- He, D., Deng, Z., Zhang, Y., Chan, S., Cheng, Y., & Guizani, N. (2020). Smart contract vulnerability analysis and security audit. *IEEE Network*, 34(5), 276–282. <https://doi.org/10.1109/MNET.001.1900656>
- Hu, K., Zhu, J., Ding, Y., Bai, X., & Huang, J. (2020). Smart contract engineering. *Electronics*, 9(12), 2042. <https://doi.org/10.3390/electronics9122042>
- Huang, R., Chen, J., Wang, Y., Bi, T., Nie, L., & Zheng, Z. (2024). An overview of Web3 technology: Infrastructure, applications, and popularity. *Blockchain: Research and Applications*, 5(1), 100173. <https://doi.org/10.1016/j.bcra.2023.100173>
- Lai, Y., Yang, J., Liu, M., Li, Y., & Li, S. (2023). Web3: Exploring decentralized technologies and applications for the future of empowerment and ownership. *Blockchains*, 1(1), 2–24. <https://doi.org/10.3390/blockchains1010002>
- McCabe, C., Mohideen, A. I. C., & Singh, R. (2024). A blockchain-based authentication mechanism for enhanced security. *Sensors*, 24(18), 5830. <https://doi.org/10.3390/s24175830>
- Tyurin, A. V., Tyulyandin, I. V., Maltsev, V. S., Kirilenko, I. A., & Berezun, D. A. (2019). Overview of the languages for safe smart contract programming. *Trudy ISP RAN/Proc. ISP RAS*, 31(3), 157–182. [https://doi.org/10.15514/ISPRAS-2019-31\(3\)-13](https://doi.org/10.15514/ISPRAS-2019-31(3)-13)

- Mohanta, B. K., Panda, S. S., & Jena, D. (2018, December). An overview of smart contract and use cases in blockchain technology. In *2018 9th International Conference on Computing, Communication and Networking Technologies (ICCCNT)* (pp. 1–4). IEEE. <https://doi.org/10.1109/ICCCNT.2018.8494045>
- Tantikul, P., & Ngamsuriyaroja, S. (2020). Exploring vulnerabilities in Solidity smart contract. In *Proceedings of the 9th International Conference on Software and Computer Applications (ICSCA)* (pp. 147–151). <https://doi.org/10.1145/3384544.3384568>
- Buterin, V. (2014). *Ethereum white paper: A next generation smart contract & decentralized application platform*. <https://ethereum.org/en/whitepaper/>
- Lee, W.-M. (2019). *Beginning Ethereum smart contracts programming: With examples in Python, Solidity and JavaScript*. Apress. <https://doi.org/10.1007/978-1-4842-5086-0>
- Seymour, T., & Goodell, G. (2024). *Custodial and non-custodial wallets*. arXiv. <https://doi.org/10.48550/arXiv.2409.15389>

Chapter 6 Appendix

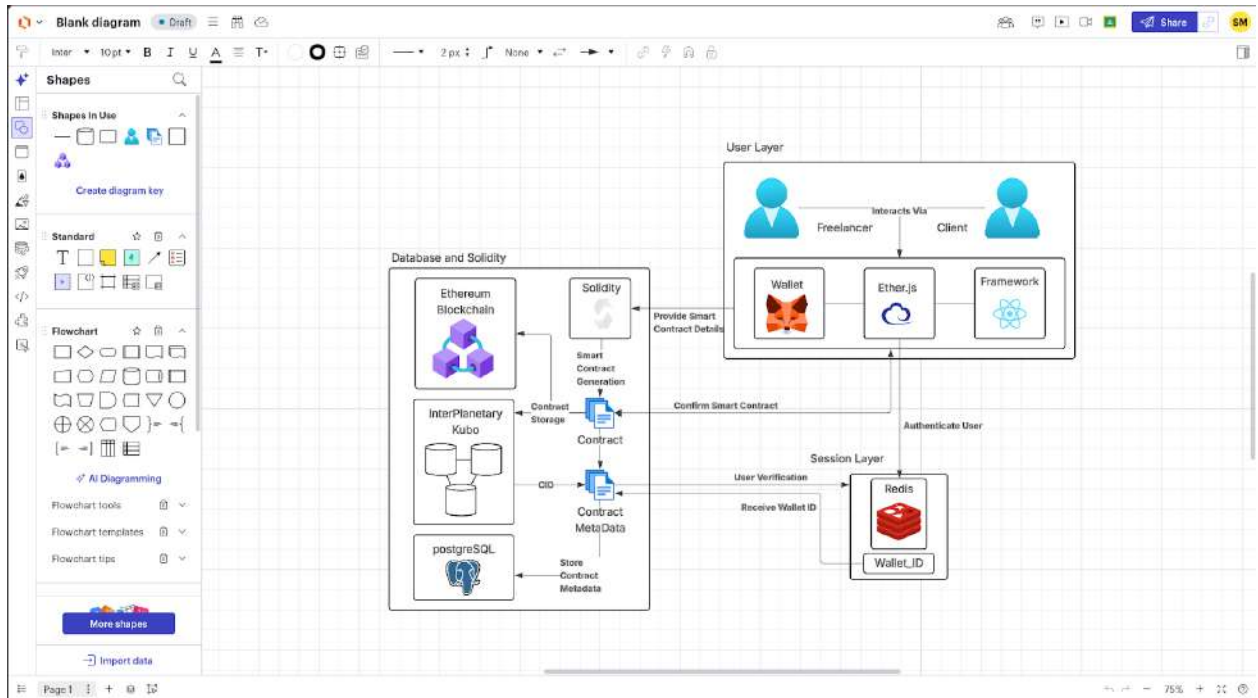


Fig 6.1: Lucidchart architecture diagram

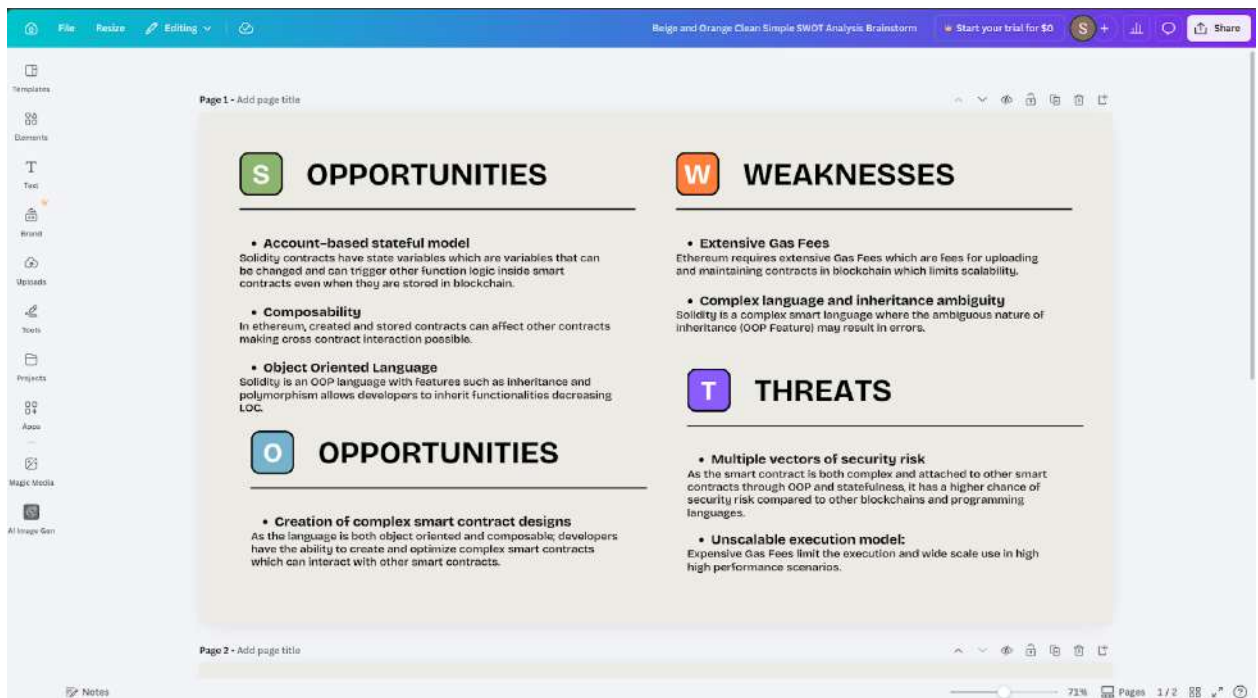


Fig 6.2: Canva SWOT analysis

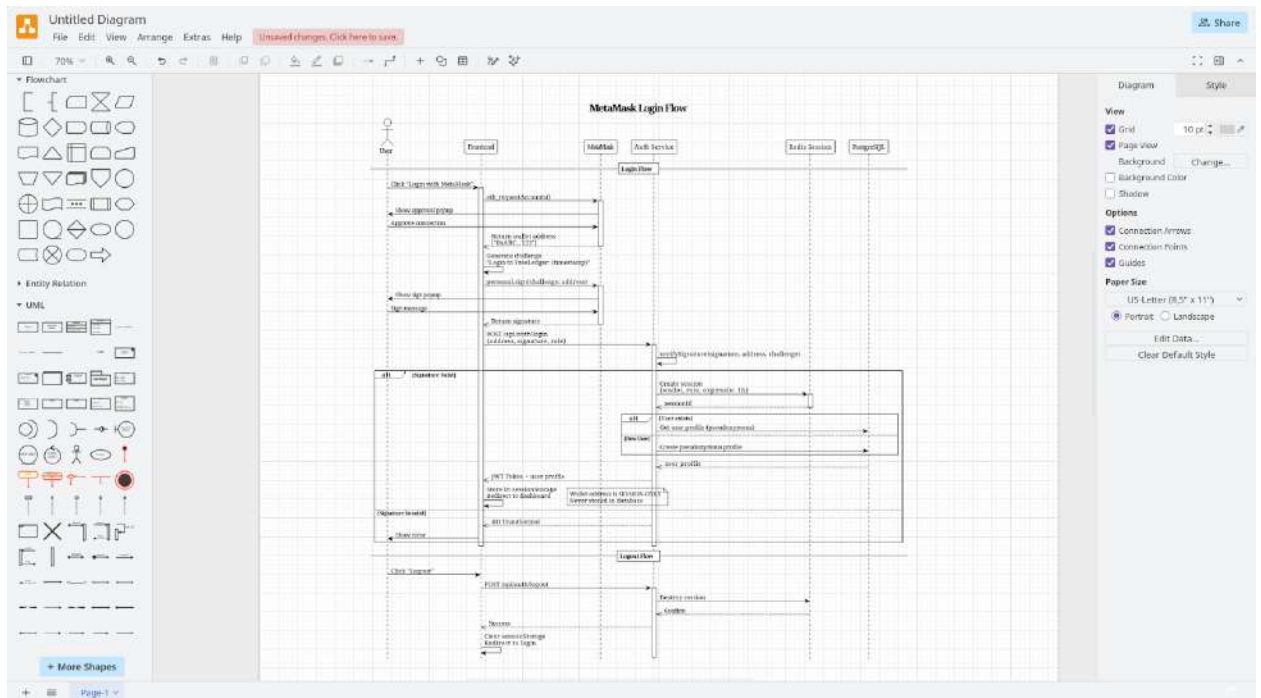


Fig 6.3: draw.io sequence diagram

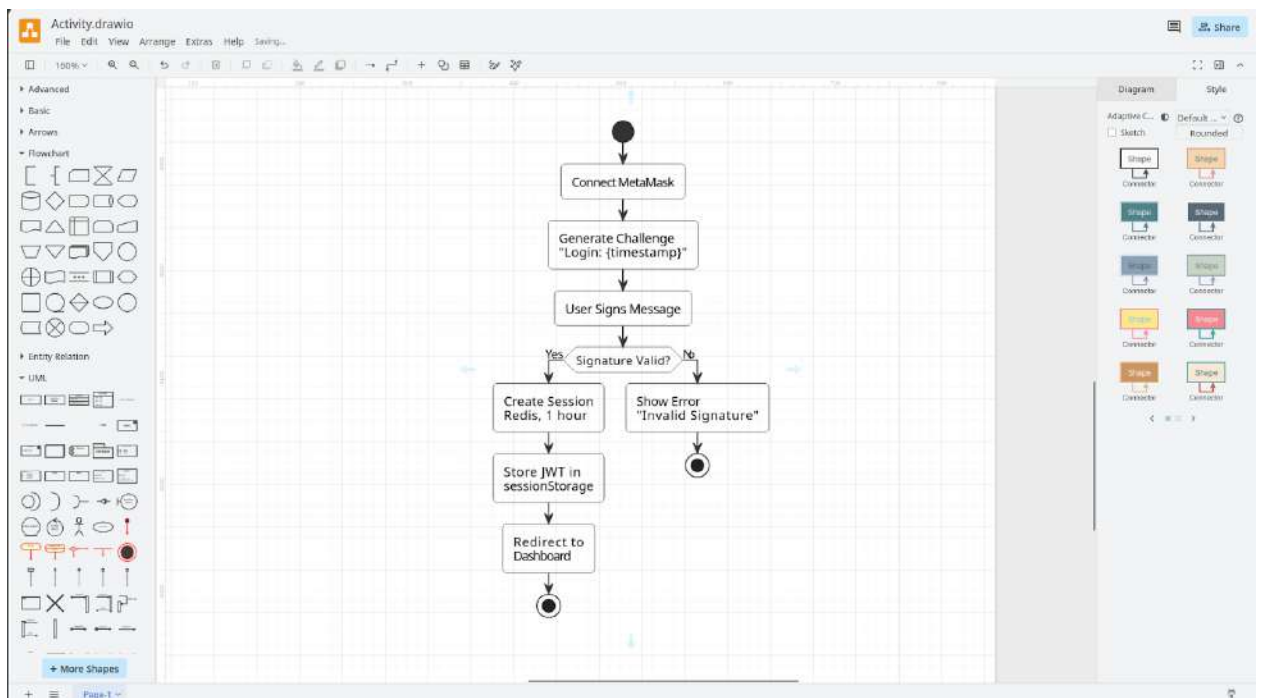


Fig 6.4: draw.io architecture diagram

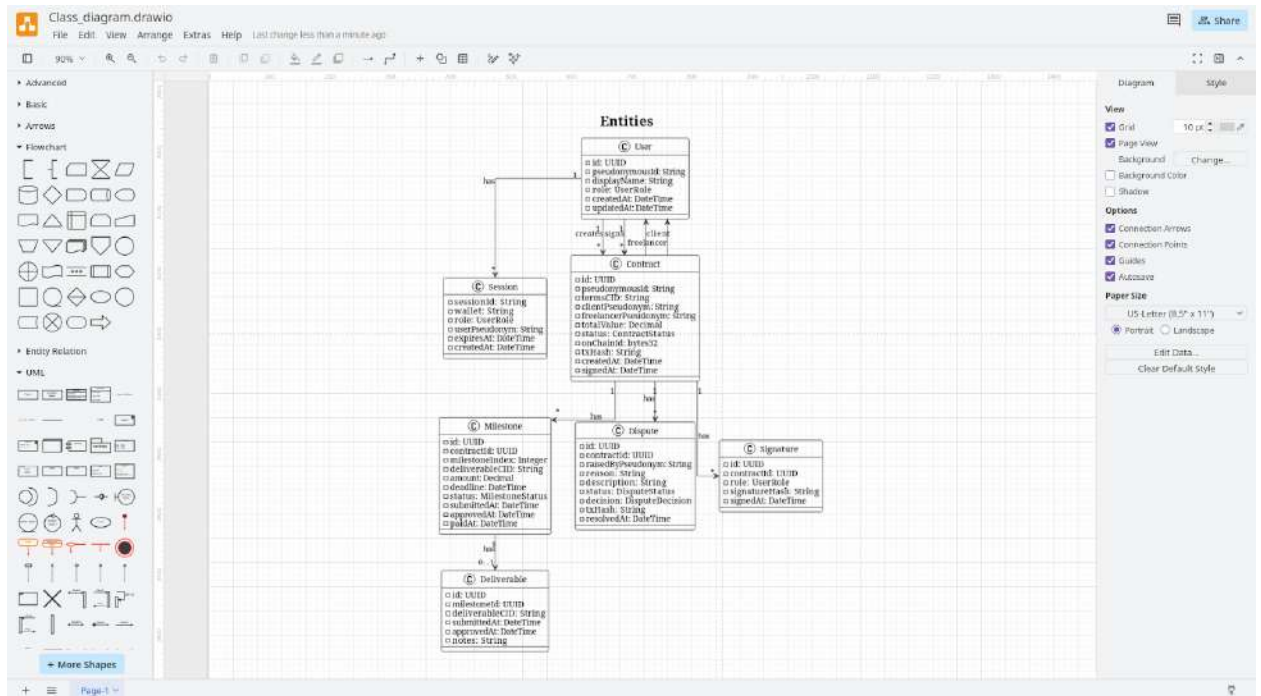


Fig 6.5: draw.io class diagram

Meeting logs -01



Bachelors of Computer Science (Hons) Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
 2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
 3. A log sheet is to be brought by the STUDENT to each supervisory session.
 4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
 5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
 6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
- The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

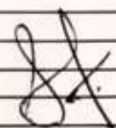
Student Name: Sarun Maharjan
Meeting No: 1

Group: 19

Date: 22/02/2026

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalaina

Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Capstone Initiation Template and Format
- Role based Literature Review
- Required Documents / Topics to be Covered

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Received Format for Literature Review
- Received specifications regarding number of papers
- Received notification regarding scope of paper to be researched
- Approved Initial Literature Review Papers

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Start Literature Review
- Continue gathering more Research Papers of various scopes/ sectors.
- Complete Literature Review for a Paper for Supervisors immediate criticism.

Note: A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Meeting minutes-02



Bachelors of Computer Science (Hons) Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
 2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
 3. A log sheet is to be brought by the STUDENT to each supervisory session.
 4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
 5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
 6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
- The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Sarun Maharjan

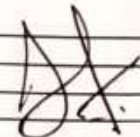
Group: 19

Date: 27/03/2026

Meeting No: 2 (Online Meeting on Google Meet [4:00-5:00 pm])

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalisina

Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Initial Literature Review
- Required Diagrams / Topics to be Covered not mentioned in instructions.

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Received criticism regarding Literature Review Language.
- SWOT analysis requires further clarification.
- Functional/ Non-Functional requirements need further sectioning and additions.
- Every individual will have a varying architecture Diagram although explanations may be similar.
- Use pre existing projects as examples in SWOT analysis
- Add mechanisms for dispute management.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Create logic / process for dispute initiation and resolution.
- Update SWOT analysis with classification of language and environment.
- Update Functional and Non-Functional Requirements.
- Create architecture diagrams, sequence diagrams based on individuals roles.

Note: A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under **NO** situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Meeting minutes-03



Bachelors of Computer Science (Hons) Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
 2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
 3. A log sheet is to be brought by the STUDENT to each supervisory session.
 4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
 5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
 6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
- The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

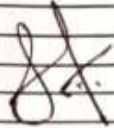
Student Name: Sarun Maharjan
Meeting No: 3

Group: 19

Date: 29/03/2026

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalaina

Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Review of updated Literature Review, SWOT, Functional vs Non-Functional Requirements.
- Additional sections for capstone guidelines.
- Review of created diagrams.

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Literature Review finalization.
- Sections such as Performance, Risk Analysis, pseudocode, may not be required.
- Capstone 2 will require further analysis of Capstone 1 project.
- Sequence, Activity, and class diagrams approval.
- Removal of functions for architecture diagram.
- Dispute logic finalization.
- Reviewed Section 1, group proposal summary.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Create sequence and activity diagrams for dispute logic.
- Explain all the created diagrams briefly.
- Create SWOT analysis on Canva.
- Update group proposal summary using only submitted group proposal.

Note: A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under **NO** situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Meeting minutes-04



Bachelors of Computer Science (Hons) Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
 2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
 3. A log sheet is to be brought by the STUDENT to each supervisory session.
 4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
 5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
 6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
- The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

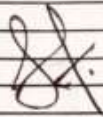
Student Name: Sarun Maharjan
Meeting No: 4

Group: 19

Date: 29/03/2026

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalina

Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Review of created dispute activity diagram.
- Creation of Overall Diagram of system.
- Questions regarding Gantt Chart and Sprint planning.

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Gantt Chart and Sprint Planning is not required for Capstone 1.
- Overall Use case diagrams should be simplified.
- For overall diagrams, only explain sections from your individual role.
- Appendix may not be required.
- Approved SWOT analysis design.
- Bibliography should include all papers even outside Literature Review.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Update overall diagrams.
- Explain individual sections from the overall diagram.
- Implement bibliography.

Note: A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under **NO** situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Meeting minutes-05



Bachelors of Computer Science (Hons) Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

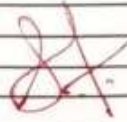
Student Name: Sarun Maharjan
Meeting No: 5

Group: 19

Date: 31/03/2026

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalina

Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Criticism regarding the overall diagram.
- Final Paper Review
- Queries regarding presentation submission

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Add Photos for proof of no AI in the appendix.
- Everybody should make their own presentation which the leader will merge later
- Only surface level technical analysis is needed for the presentation.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Final Updates to the content of Research Paper.
- Start creating presentation slides

Note: A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

Meeting minutes-06



Bachelors of Computer Science (Hons) Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2.
 2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
 3. A log sheet is to be brought by the STUDENT to each supervisory session.
 4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
 5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
 6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.
- The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Sarun Maharjan
Meeting No: 6

Group: 19

Date: 01/04/2026

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalaina

Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- AI checking for uploaded content.
- Template for the final submission of the pdf.
- Presentation Template queries.

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- 0 % AI was noted down with 20 % plagiarism.
- A template for the format was shown and supervisor advised implementing it.
- The Presentation Template was shown but the supervisor advised not to strictly follow the template.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Final Updates to the Research Paper with required Formatting.
- Creating the presentation with template in mind.

Note: A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under **NO** situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.